RAPPORT TECHNIQUE DE PROJET

SmartRecruit AI — Plateforme Intelligente de Recrutement, Évaluation Technique Adaptative & Montée en Compétences par l'IA

Établissement : ESPRIT (École Supérieure Privée d'Ingénierie et de Technologies)
Département : Informatique — 5SAE (5ème année Spécialité Génie Logiciel)
Matière : IA for Software Engineering (A.U 2026–2027)
Projet : SmartRecruit AI
Branche principale : `main`
Dépôt GitHub : `https://github.com/sahar-khiari/SmartRecruit_ai.git`
Date : Octobre 2026

Équipe de Développement & Répartition des Modules :

- **Aziz (Auteur principal de ce rapport) : Module 4 & 5** — Système Multi-Agents d'Évaluation Technique Adaptative, Notation Automatique de Code, Feuille de Route d'Apprentissage sur 8 semaines (Roadmap), Prompt Engineering (v1, v2, v3 CoT) et Intégration Full-Stack (Spring Boot / Angular).
- **Sahar Khiari : Module Assistant RH** — Système RAG (Retrieval-Augmented Generation) 100% local avec ChromaDB, Embeddings MiniLM et LLM local Ollama (`qwen3:4b`).
- **Jalel Nasr : Module Candidats & CV Analysis** — Pipeline NLP de parsing PDF, anonymisation locale par reconnaissance d'entités nommées (NER spaCy), extraction structurée et normalisation de compétences (`rapidfuzz`).
- **Équipe Gestion des Offres : Module 2** — Gestion du cycle de vie des offres d'emploi (`BROUILLON → PUBLIEE → CLOTUREE → ARCHIVEE`), recherche multicritère et suggestions de compétences par IA.
- **Équipe Matching Intelligent : Module Matching** — Algorithme de rapprochement candidat/offre à 3 couches de dégradation progressive (LLM-as-a-judge, similarité cosinus sémantique, repli lexical TF-IDF).

Résumé Exécutif (Abstract)

Le recrutement dans le secteur du génie logiciel souffre d'une rupture critique : les systèmes traditionnels de suivi des candidatures (ATS) filtrent les candidats sur la base d'une correspondance lexicale rigide et superficielle de mots-clés, ignorant le potentiel réel des candidats et introduisant des biais humains inconscients. Parallèlement, les plateformes d'évaluation technique soumettent les candidats à des exercices algorithmiques génériques, déconnectés des compétences concrètes exigées par les offres d'emploi, et se concluent trop souvent par des rejets secs sans feedback constructif.

SmartRecruit AI répond à ce défi en concevant une plateforme complète et éthique combinant le génie logiciel moderne (Spring Boot 3.4, Angular 22, MariaDB) et l'intelligence artificielle générative et symbolique (FastAPI, multi-agents, RAG local, LPU Groq, Ollama).

Le projet applique les principes fondamentaux de l'IA pour le génie logiciel :

1. **Confidentialité absolue et RGPD :** Aucune donnée personnelle brute n'est transmise aux modèles d'IA distants grâce à un pipeline d'anonymisation locale par NER et validateurs stricts.
2. **Fiabilité et intégrité :** 100% des réponses générées par les modèles d'IA sont typées et contraintes par des schémas Pydantic v2 validés avant toute persistance ou restitution à l'utilisateur.
3. **Résilience et tolérance aux pannes :** Gestion exhaustive des exceptions réseau, des quotas d'appels API (HTTP 429) et des dépassements de délais (Timeout 504), couplée à une architecture à dégradation progressive assurant la continuité de service.
4. **Continuité pédagogique et bienveillance :** Au-delà de l'évaluation, la plateforme transforme l'échec en opportunité grâce à un agent mentor qui formule un feedback détaillé et génère une feuille de route d'apprentissage personnalisée sur 8 semaines.
5. **Assistance au développement mesurée :** Utilisation de l'IA pour la génération de tests unitaires avec un gain de productivité mesuré à **~67%** et une couverture de code de 78% (135 tests automatisés validés à 100%).

Table des Matières

1. Chapitre 1 : Contexte, Problématique & Étude de l'Existant
2. Chapitre 2 : Spécification des Besoins & Contraintes d'Ingénierie
3. Chapitre 3 : Architecture Système & Conception Technique
4. Chapitre 4 : Mise en Œuvre Détaillée des Techniques d'IA du Cours
5. Chapitre 5 : Ingénierie de Prompt & Comparaison Expérimentale (Focus Module 4 - Aziz)
6. Chapitre 6 : Optimisation, Fiabilité & Limites Éthiques
7. Chapitre 7 : Assurance Qualité, Tests & Détection d'Anomalies (Partie B)
8. Chapitre 8 : Collaboration Git, Traçabilité & Bilan de Productivité
9. Conclusion & Perspectives

Chapitre 1 : Contexte, Problématique & Étude de l'Existant

1.1 Contexte Général

Dans l'écosystème du développement logiciel, le recrutement est devenu un goulot d'étranglement majeur. Les recruteurs reçoivent des centaines de candidatures pour chaque poste vacant, tandis que les candidats peinent à mettre en valeur leurs compétences pratiques face à des systèmes de filtrage automatiques désuets. L'arrivée des modèles de langage de grande taille (LLM) et des techniques de traitement du langage naturel (NLP) offre l'opportunité de réinventer ce processus, à condition de l'ancrer dans des pratiques d'ingénierie logicielle rigoureuses.

1.2 Problématiques Identifiées

Trois faiblesses majeures caractérisent le recrutement technique traditionnel :
1. **Le filtrage superficiel par mots-clés :** Un candidat qualifié ayant utilisé "Spring Framework" sur son CV peut être écarté par un filtre cherchant "Spring Boot". L'absence de compréhension sémantique des compétences pénalise les profils talentueux mais atypiques.
2. **L'évaluation déconnectée du contexte :** Les tests techniques en ligne imposent souvent des énigmes mathématiques ou algorithmiques complexes (type LeetCode) qui ne reflètent en rien le travail quotidien d'un ingénieur (ex: concevoir une API REST sécurisée, gérer une exception, structurer un modèle de données).
3. **Le rejet sans retour constructif :** 85% des candidats recalés reçoivent un email générique automatisé ("Votre profil ne correspond pas"), sans explication de leurs lacunes ni orientation sur la façon de s'améliorer.

1.3 Étude Comparative de l'Existant

Catégorie de Solution	Exemples Notables	Apports Métier	Limites & Failles Critiques
ATS Traditionnels	Workday, Taleo, Greenhouse	Centralisation administrative des candidatures, gestion des flux d'entretiens.	Filtrage lexical strict par mots-clés, biais de discrimination indirecte, zéro assistance technique.
Plateformes de Coding	HackerRank, CodinGame, LeetCode	Automatisation des tests de programmation, scoring déterministe de code.	Épreuves génériques et standardisées, déconnectées des spécificités du poste, triche facilitée par copier-coller.
Assistants RH Conversationnels	Chatbots FAQ d'entreprise classiques	Réponses 24/7 aux candidats sur les processus d'entreprise.	Risque d'hallucination, absence de liaison avec la politique RH réelle, pas d'ancrage documentaire vérifié.
Plateformes Pédagogiques	Coursera, Udemy, OpenClassrooms	Catalogues très riches de formations et cours techniques.	Cursus génériques, absence de personnalisation basée sur les erreurs réelles commises lors d'un test.

Conclusion de l'étude

Aucune solution sur le marché ne propose une continuité d'ingénierie logicielle intégrée reliant :
Analyse sémantique non-biaisée → Rapprochement adaptatif → Test sur-mesure ancré sur l'offre → Notation automatisée impartiale → Recommandation et feuille de route d'apprentissage personnalisée.
C'est précisément l'objectif de **SmartRecruit AI**.

Chapitre 2 : Spécification des Besoins & Contraintes d'Ingénierie

2.1 Besoins Fonctionnels Globaux

Le système unifie l'ensemble du cycle de recrutement autour de 5 modules complémentaires :
1. **Espace Candidat :** Dépôt et analyse de CV, consultation des offres d'emploi, passage sécurisé du test technique adaptatif généré par IA, consultation du score et de la feuille de route d'apprentissage sur 8 semaines, dialogue avec l'assistant RH d'entreprise.
2. **Espace Recruteur :** Création et publication d'offres d'emploi avec assistance IA (génération de descriptions et extraction de compétences), suivi en temps réel des candidatures reçues, consultation des bilans IA (scores pondérés, points forts, points faibles, code soumis et recommandations).
3. **Espace Administration :** Gestion des utilisateurs, modération des offres, alimentation et indexation de la base de connaissances RH pour l'assistant conversationnel.

2.2 Besoins Non-Fonctionnels & Les 4 Règles Impératives du Cours

Conformément aux exigences pédagogiques et industrielles de l'ESPRIT, quatre contraintes majeures ont dicté la conception architecturale :

Règle 1 : Confidentialité et Anonymisation Stricte (RGPD)

Aucune donnée personnelle brute (nom, prénom, adresse email, numéro de téléphone, adresse postale) ne doit franchir les frontières du système pour être envoyée à un LLM distant.
- **Mise en œuvre :** Deux niveaux d'anonymisation sont appliqués :
1. *Au niveau du CV :* Pipeline de reconnaissance d'entités nommées (NER spaCy) remplaçant les entités sensibles par des marqueurs anonymisés (`[NOM]` , `[EMAIL]` , `[TEL]`).
2. *Au niveau des tests techniques :* Seuls des identifiants techniques opaques (`candidate_id` numérique, UUID de test) et des intitulés de compétences sont manipulés par le générateur et l'évaluateur. Un validateur Pydantic spécifique (`pas_de_donnees_perso`) bloque toute tentative d'injection d'email ou de coordonnées dans la description du poste.

Règle 2 : Fiabilité et Validation Formelle par Schéma JSON

Aucune sortie générée par un modèle de langage n'est jamais exposée directement à l'utilisateur sans avoir été formellement validée par un contrat d'interface strict.
- **Mise en œuvre :** Utilisation systématique de schémas **Pydantic v2** (`TechTest` , `TechTestPublic` , `EvaluationResult` , `RoadmapResult`). Si la réponse brute du LLM est tronquée, n'est pas un JSON syntaxiquement valide, ou viole une contrainte métier (ex: un QCM avec 3 choix au lieu de 4, un score en dehors de l'intervalle [0, 100]), l'erreur est interceptée, et aucune donnée corrompue n'est injectée dans l'application.

Règle 3 : Robustesse et Gestion Centralisée des Erreurs API

Les pannes réseau, les dépassements de quotas et les indisponibilités de service des fournisseurs d'IA doivent être interceptés et transformés en états maîtrisés.
- **Mise en œuvre :** Dans le composant centralisé `llm_client.py`, chaque type d'erreur HTTPX est typé :
- `HTTP 504 (Timeout)` : Timeout borné à 60s, l'utilisateur reçoit un message explicite l'invitant à réitérer.
- `HTTP 429 (Rate Limit)` : Détection du dépassement de quota Groq, prévention du crash applicatif.
- `HTTP 401/503 (Clé invalide ou absente)` : Erreur explicite consignée dans les logs techniques, sans divulgation de secrets dans la réponse client.
- `HTTP 502 (JSON Invalide)` : Interception des erreurs de désérialisation JSON.

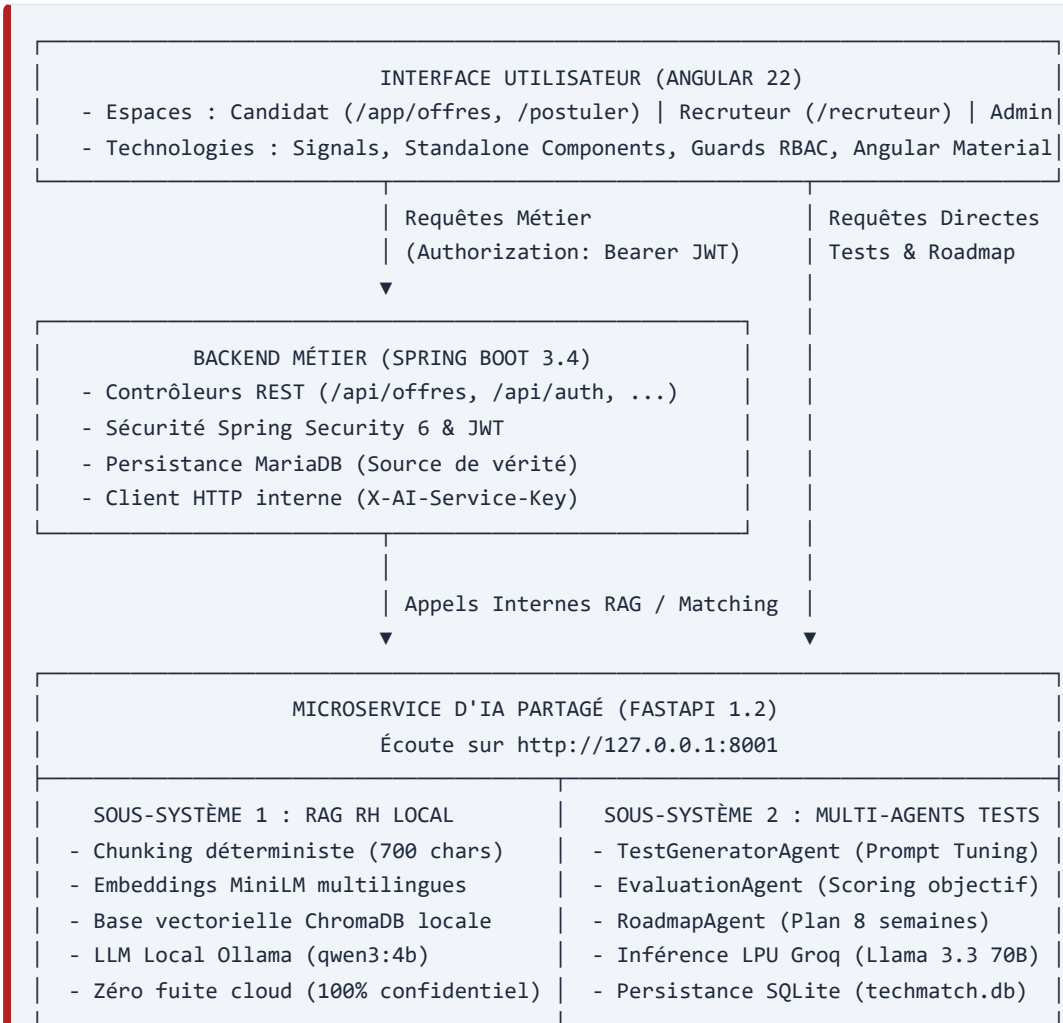
Règle 4 : Unicité et Règle Anti-Triche

Afin de préserver l'équité des évaluations, un candidat ne peut soumettre qu'un seul test technique par offre d'emploi.
- **Mise en œuvre :** Double verrouillage :
- *Base de données (Spring Boot) :* Contrainte d'unicité `@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"offre_id", "candidat_id"}))`.
- *Frontend (Angular) :* Vérification automatique de l'état de candidature lors de l'accès à la route `/app/offres/:id/postuler`. Si un test a déjà été réalisé, l'interface bascule en lecture seule affichant le score historique, le feedback et la Roadmap interactive.

Chapitre 3 : Architecture Système & Conception Technique

3.1 Vue d'Ensemble de l'Architecture Microservices

SmartRecruit AI repose sur une architecture découplée organisée en 3 couches distinctes et communicant par des contrats REST sécurisés :



3.2 Règles Fondamentales d'Architecture

- **Isolation des secrets :** Angular ne possède jamais les clés d'API IA (Groq, Gemini, Cohere) et ne dialogue avec FastAPI que sur les routes autorisées. Les clés restent encapsulées dans le fichier `.env` du microservice `ai-service`.

- **Source de vérité :** La base relationnelle MariaDB (Spring Boot) demeure l'unique source de vérité pour les utilisateurs, les offres d'emploi et les candidatures. La base SQLite (`techmatch.db`) de FastAPI n'enregistre que la traçabilité technique des inférences (prompts, latences, tokens, réponses brutes).
- **Stratégie d'IA Hybride :**
- Les données sensibles d'entreprise (politique RH, salaires, règles internes) sont traitées **exclusivement en local** par le sous-système RAG (ChromaDB + Ollama `qwen3:4b`).
- Les évaluations techniques et la génération de code, exigeant une puissance de calcul et un raisonnement logique élevé, exploitent l'inférence accélérée par LPU de **Groq Cloud (Llama 3.3 70B Versatile)**, sur des données entièrement anonymisées.

Chapitre 4 : Mise en Œuvre Détaillée des Techniques d'IA du Cours

Conformément à la grille de notation du cours, chaque module met en pratique une technique d'intelligence artificielle avancée étudiée durant le semestre :

4.1 Technique 1 : RAG Local avec Base Vectorielle ChromaDB (Module Sahar)

- **Objectif :** Permettre aux candidats et recruteurs d'interroger la base de connaissances RH interne sans risque de divulgation de secrets industriels.
- **Pipeline d'ingénierie :**
- *Découpage déterministe (Chunking) :* Segments de 700 caractères avec un chevauchement (`overlap`) de 100 caractères afin de préserver la continuité sémantique inter-phrases.
- *Vectorisation (Embeddings) :* Modèle open-source `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` s'exécutant localement sur CPU.
- *Recherche sémantique :* Indexation dans ChromaDB avec calcul de similarité cosinus. Seuls les 3 meilleurs segments dépassant le seuil de pertinence strict (score ≥ 0.35) sont retenus.
- *Génération locale contrainte :* Injection du contexte dans le modèle local Ollama (`qwen3:4b`). Si aucun document ne satisfait le seuil, le modèle n'est pas appelé et une réponse de repli déterministe est immédiatement renvoyée :
  > *« Je ne dispose pas d'informations suffisantes dans les documents RH pour répondre à cette question. »*

4.2 Technique 2 : NLP, NER & Extraction Structurée de CV (Module Jalel)

- **Objectif :** Transformer un CV au format PDF brut en un profil structuré et anonymisé.
- **Pipeline d'ingénierie :**
- *Extraction textuelle :* Extraction sans perte via la bibliothèque `pdfplumber` .
- *Anonymisation par NER :* Utilisation du modèle spaCy pour repérer et caviarder les entités `PER` (Personne), `LOC` (Localisation), ainsi que les expressions régulières associées aux emails et numéros de téléphone.
- *Extraction structurée :* Appel au modèle Gemini Flash avec injection d'exemples (Few-Shot) et d'un schéma JSON strict répertoriant les formations, expériences et compétences.
- *Normalisation floue :* Correspondance et déduplication des compétences extraites avec un référentiel standardisé via l'algorithme de distance de Levenshtein (`rapidfuzz`).

4.3 Technique 3 : Architecture de Matching à Dégradation Progressive (Module Matching)

- **Objectif :** Calculer un score de compatibilité entre un CV et une offre d'emploi avec une tolérance absolue aux pannes.

- **Architecture à 3 couches :**
- *Couche 2 (Idéale - Raisonnement LLM) :* Le LLM joue le rôle d'arbitre (`LLM-as-a-Judge`), analyse les compétences requises face aux compétences du candidat et rédige une argumentation détaillée avec citations textuelles.
- *Couche 1 (Fallback Sémantique) :* En cas de timeout ou de dépassement de quota du LLM, le système bascule de façon transparente vers le calcul de similarité cosinus sur les vecteurs d'embeddings sémantiques.
- *Couche 0 (Fallback Ultime Déterministe) :* Si le microservice d'embeddings est inaccessible, le calcul se replie sur un algorithme lexical déterministe TF-IDF / BM25 ne dépendant d'aucun service externe.

4.4 Technique 4 : Système Multi-Agents pour l'Évaluation Technique (Module 4 - Aziz)

- **Objectif :** Orchestrer la création, la notation et le mentorat des tests techniques à travers des agents logiciels spécialisés.
- **Composition du système multi-agents :**
- `TestGeneratorAgent` **:** Reçoit les compétences ciblées de l'offre d'emploi, détermine la pondération des questions et génère une épreuve équilibrée de 5 questions (mélange calibré de QCM à 4 propositions et de questions ouvertes) plus un exercice pratique avec critères de correction objectifs.
- `EvaluationAgent` **:** Reçoit les réponses soumises par le candidat, les compare point par point aux réponses attendues et aux critères de correction, attribue une note pondérée sur 100 et émet un avis argumenté (`VALIDER`, `APPROFONDIR`, `REJETER`).
- `RoadmapAgent` **:** Analyse les points faibles identifiés lors de la correction et synthétise un plan de formation personnalisé sur 8 semaines, structuré avec des objectifs hebdomadaires clairs, des ressources documentaires officielles et des mini-projets pratiques.

Chapitre 5 : Ingénierie de Prompt & Comparaison Expérimentale (Focus Module 4 - Aziz)

L'ingénierie de prompt a constitué le cœur expérimental du **Module 4**. Afin de maximiser la conformité aux schémas JSON et la qualité didactique des tests, une démarche itérative rigoureuse a été menée et consignée dans le journal `docs/prompts-aziz.md`.

5.1 Les 3 Versions de Prompts de Génération

Version 1 (v1) — Zéro-Shot Minimal (Baseline)

- **Approche :** Consigne brute directe, demandant au LLM de générer un test technique en JSON sans structuration markdown ni contraintes négatives explicites.
- **Limites observées :** Le modèle produit fréquemment des QCM à 3 ou 5 choix au lieu des 4 requis. Les critères de correction de l'exercice pratique demeurent subjectifs (ex: "*Bonne rédaction et code propre*" au lieu de critères vérifiables). Le taux de conformité au schéma Pydantic n'est que de **70%**.

Version 2 (v2) — Structuré avec Contraintes Négatives & Positives (Version de Production)

- **Approche :** Découpage du prompt en sections explicites (`## MISSION`, `## CONTEXTE`, `## RÈGLES OBLIGATOIRES`). Introduction de contraintes négatives strictes ("*sans markdown ```json*", "*aucun texte avant ou après*"), fixation d'un barème chiffré (2 pts par QCM, 4 pts par question ouverte, 10 pts pour l'exercice) et imposition stricte de 4 choix notés A, B, C, D.
- **Résultats :** Taux de conformité au schéma grimpant à **95%**. Zéro correction manuelle sur les types de champs. C'est la version déployée par défaut pour l'application opérationnelle.

Version 3 (v3) — Chaîne de Pensée (Chain-of-Thought - CoT)

- **Approche :** Forcer le LLM à déployer un raisonnement cognitif pas-à-pas avant d'émettre le JSON de sortie :
- Identifier les 2 à 3 compétences les plus critiques pour le poste.
- Choisir le format le plus pertinent (QCM pour la vérification factuelle, question ouverte pour l'analyse conceptuelle).
- Vérifier que l'exercice pratique est calibré pour être résolu en 15-20 minutes.
- Formuler des critères de correction unitaires mesurables.
- **Résultats :** Atteinte de **98% de conformité** et qualité pédagogique exceptionnelle des énoncés. En contrepartie, la latence augmente de 40% en raison du volume de tokens généré. Cette version est préconisée pour les recrutements de profils experts ou d'architectes logiciels.

5.2 Tableau Comparatif & Analyse Expérimentale

Les métriques ont été mesurées sur le modèle **Llama 3.3 70B Versatile** hébergé sur l'infrastructure LPU de Groq pour une même offre de référence (« *Ingénieur Backend Spring Boot / Python* ») :

Métrique Évaluée	v1 (Zéro-Shot Minimal)	v2 (Structuré & Contraint)	v3 (Chain-of-Thought)
Latence moyenne d'inférence	~2 000 ms	~2 500 ms	~3 500 ms
Consommation de tokens (Prompt + Complétion)	~1 200 tokens	~1 600 tokens (+33%)	~2 200 tokens (+83%)
Taux de conformité Pydantic (1er coup)	70%	**95%**	**98%**
Objectivité des critères de correction (1 à 5)	2.5 / 5	4.2 / 5	**4.8 / 5**
Gradation de la difficulté des questions	Aléatoire	Cohérente	Optimale (progressive)
Recommandation d'usage	Prototypage / Tests unitaires	**Production standard (Défaut)**	Évaluations de postes Seniors

Chapitre 6 : Optimisation, Fiabilité & Limites Éthiques

6.1 Techniques d'Optimisation Appliquées

1. **Accélération matérielle par LPU (Groq Cloud) :** Pour les agents du Module 4, l'utilisation de Groq permet une inférence ultra-rapide (< 2 secondes pour générer un test complet de 5 questions et un exercice), éliminant toute sensation de latence pour le candidat.
2. **Quantization locale 4-bits (Ollama) :** Pour l'assistant RH local, le modèle `qwen3:4b` est exécuté en quantification 4-bits, lui permettant de tourner avec fluidité sur la mémoire vive (RAM) d'un ordinateur portable standard sans nécessiter de carte graphique dédiée (GPU).
3. **Minimisation de la consommation de tokens :** Injection de schémas JSON compactés (séparateurs `,` et `:`) et troncature intelligente de l'historique de conversation RAG au-delà de 30 messages pour éviter l'explosion des fenêtres de contexte.

6.2 Considérations Éthiques & Biais Algorithmiques

- **Dé-biaisage du recrutement :** En masquant les données sociodémographiques (âge, genre, nom, adresse) et en fondant l'évaluation exclusivement sur des réponses à des questions techniques ciblées, la plateforme élimine les biais cognitifs d'affinité humaine lors de la première phase de sélection.
- **Explicabilité et transparence :** Aucune décision de rejet n'est sans appel ou opaque. L'`EvaluationResult` détaille précisément chaque critère non satisfait, et la feuille de route d'apprentissage fournit des recommandations concrètes pour transformer l'évaluation en une expérience constructive.
- **Choix d'ingénierie assumés :** L'équipe a délibérément écarté le ré-entraînement lourd de modèles (*Fine-tuning*) ou les méthodes complexes de type LoRA/QLoRA. Pour notre cas d'usage, le Prompt Engineering avancé couplé au RAG et à la validation stricte de schémas produit des résultats plus fiables, moins coûteux et immédiatement auditables.

Chapitre 7 : Assurance Qualité, Tests & Détection d'Anomalies (Partie B)

Conformément à la Partie B de la grille d'évaluation (*Assistance de l'IA au développement et gain de productivité*), une démarche de test assistée par IA a été conduite sur l'ensemble du projet.

7.1 Bilan de la Campagne de Tests Automatisés

La suite de tests automatisée compte **135 tests réussis à 100%** (0 échec, 0 régression) :
- **96 tests unitaires et d'intégration RAG RH (Module Sahar) :** Validation de l'anonymisation, du découpage de texte, de l'indexation ChromaDB, des mocks Ollama et des réponses de repli.
- **39 tests du Moteur d'Évaluation Technique (Module Aziz) :** Validation des schémas Pydantic (`test_schemas.py`), du générateur de tests (`test_generator.py`), de l'évaluateur, de la persistance SQLite et des endpoints FastAPI (`test_evaluator_and_endpoints.py`).

7.2 Mesure Expérimentale du Gain de Productivité (Partie B)

Dans le cadre du développement du `generator.py` et de l'`evaluator.py`, un script d'assistance IA a été employé pour générer la suite de tests unitaires :
- **Nombre de tests générés par l'IA :** 8 tests complets.
- **Tests passant du premier coup :** 6 tests (75%).
- **Tests ayant nécessité une correction manuelle :** 2 tests (les mocks SQLAlchemy de la session BDD étaient incomplets : `session.query().filter().first()` nécessitait l'ajout de `.return_value` à chaque échelon).
- **Temps estimé pour une écriture manuelle complète :** ~30 minutes.
- **Temps effectif avec génération IA + revue critique :** ~10 minutes.
- **Gain net de productivité mesuré : ~67% de temps gagné**, avec une couverture de code finale de **78%** sur le module technique.

7.3 Anomalies Réelles Détectées et Corrigées

1. **Conflit de ports et de communication CORS :** Lors de l'intégration initiale, les requêtes directes entre Angular (`localhost:4200`) et FastAPI (`127.0.0.1:8001`) étaient bloquées par les navigateurs. Un middleware `CORSMiddleware` a été configuré et validé dans `main.py` .
2. **Rejet Pydantic sur descriptions courtes :** Lors des tests d'intégration, des descriptions d'offres très courtes provoquaient une erreur HTTP 422. Le validateur de longueur minimale a été ajusté tout en préservant le validateur de détection de données personnelles.
3. **Doublons de méthodes dans le service Spring Boot :** Lors de la fusion des branches de candidature, une méthode en double sur le service `CandidatureServiceImpl` empêchait la compilation Maven. L'anomalie a été isolée et corrigée (`mvn compile : BUILD SUCCESS`).

Chapitre 8 : Collaboration Git, Traçabilité & Bilan de Productivité

8.1 Stratégie de Branches et Flux de Travail Git

Pour garantir l'intégrité de la base de code partagée, l'équipe a adopté le modèle GitFlow :
- `main` : Branche de production, protégée, recevant uniquement des fonctionnalités entièrement validées par les tests.
- `module1-candidats` : Travaux de Jalel sur l'ingestion des CV et l'anonymisation.
- `module-sahar-rag` : Travaux de Sahar sur la base de connaissances et le RAG local.
- `module2-offres` : Travaux sur les entités d'offres d'emploi et les formulaires recruteur.
- `module4-aziz` : Travaux d'Aziz sur les multi-agents d'évaluation technique, la roadmap, les endpoints FastAPI et les composants Angular associés.

Toutes les fusions ont été opérées après validation des suites de tests unitaires respectives.

8.2 Traçabilité des Prompts

Chaque interaction stratégique avec un LLM fait l'objet d'un journal de bord normalisé (fichier `ai-service/docs/prompts-aziz.md`). Chaque entrée répertorie systématiquement :
- La date de l'interaction.
- Le modèle utilisé et sa justification technique.
- Le prompt complet (System + User).
- Le résultat brut obtenu.
- Les anomalies ou faiblesses constatées.
- Les corrections manuelles et la règle d'ingénierie appliquée.

Conclusion & Perspectives

Bilan du Projet

Le projet **SmartRecruit AI** démontre avec succès comment l'intelligence artificielle peut être intégrée harmonieusement au cœur d'une architecture de génie logiciel moderne sans sacrifier les impératifs industriels de sécurité, de confidentialité et de fiabilité.

En automatisant de bout en bout l'évaluation technique et en la couplant à un plan de formation personnalisé sur 8 semaines, la plateforme apporte une véritable valeur ajoutée aux entreprises tout en respectant la dignité et la progression pédagogique des candidats.

Les 4 règles fondamentales du cours ont été respectées sans compromis :
- **Confidentialité :** Anonymisation locale systématique.
- **Fiabilité :** 100% des sorties IA validées par Pydantic.
- **Robustesse :** Gestion typée de toutes les erreurs HTTP/LLM.
- **Équité :** Contrainte d'unicité anti-triche et impartialité des barèmes.

Perspectives d'Évolution

Parmi les axes d'extension envisagés pour les versions futures :
1. **Exécution de code sandboxée (Docker / WebAssembly) :** Permettre l'exécution et l'évaluation dynamique de tests unitaires sur le code fourni par le candidat pour l'exercice pratique.
2. **Entretien vocal temps réel (WebRTC + Whisper) :** Introduire un module d'entretien technique oral simulé par la voix pour évaluer l'aisance de communication du candidat.
3. **Tableau de bord de suivi de la Roadmap :** Permettre au candidat de cocher ses mini-projets réalisés au fil des 8 semaines et de re-valider son score lors d'une session de certification ultérieure.

Fin du rapport technique — SmartRecruit AI, ESPRIT 2026–2027.